

PERTEMUAN VIII

NEURAL NETWORK



8.1 TUJUAN PEMBELAJARAN

- A. Mahasiswa memahami konsep Multilayer Perceptron (MLP) dan mampu menerapkannya pada teknik klasifikasi dan regresi
- B. Mahasiswa mampu menganalisis hasil klasifikasi menggunakan MLP dengan memanfaatkan metrik evaluasi dan pengujian model.
- C. Mahasiswa mampu menganalisis hasil regresi menggunakan MLP dengan membandingkan hasil prediksi dengan data asli.

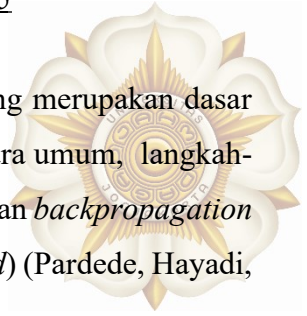
8.2 DASAR TEORI

A. Neural Network

Neural network atau jaringan syaraf tiruan adalah jaringan atau jaring laba-laba neuron, merupakan tiruan buatan dari otak manusia yang menghubungkan lapisan input ke lapisan *output*. *Neural network* adalah Teknik pemodelan yang sangat canggih dan dapat memodelkan fungsi yang sangat kompleks. Menggunakan jaringan syaraf tiruan, dimungkinkan untuk memprediksi harapan masa depan, mengklasifikasikan nilai data ke dalam kelompok yang memungkinkan keputusan dibuat menggunakan mesin yang cerdas secara artifisial. Seperti manusia, jaringan saraf tiruan dilatih oleh supervisor sesuai dengan paradigma pembelajaran yang diawasi. *Neural network* adalah bagian dari kecerdasan buatan dan pembelajaran mesin. Pembelajaran mendalam menjadi lebih maju dengan bertambahnya data. Oleh karena itu, jaringan saraf menyesuaikan kinerjanya menjadi lebih besar saat mereka tumbuh lebih besar dan menangani arus informasi yang sangat besar (Hukubun, 2022).

B. Multilayer Perceptron

Multi Layer Perceptron (MLP) salah satu algoritma *deep learning*, sudah banyak digunakan dalam penelitian masalah klasifikasi karena memiliki kelebihan dibandingkan metode klasifikasi konvensional lain. MLP memiliki kelebihan seperti mampu mengadaptasikan dirinya dengan data, dapat memperkirakan hubungan antar keanggotaan kelas dengan atribut dari objek, lebih reliabel terhadap *noise*



dalam data, dan mampu menghitung nilai probabilitas posterior, yang merupakan dasar untuk membangun aturan klasifikasi dan analisis statistika. Secara umum, langkah-langkah dalam pelatihan algoritma *multilayer perceptron* menggunakan *backpropagation* untuk mengubah nilai bobot-bobotnya dalam arah mundur (*backward*) (Pardede, Hayadi, & Iskandar, 2022).

C. Backpropagation Learning

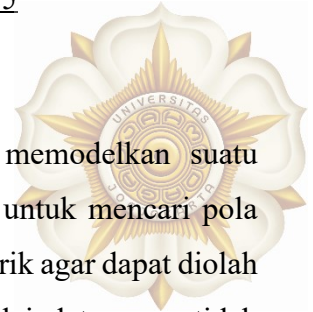
Backpropagation adalah algoritma pembelajaran yang digunakan untuk mengoptimalkan bobot (weights) neuron dalam jaringan syaraf tiruan. Prinsipnya adalah menghitung error antara output jaringan dan target, kemudian menyebarkan error tersebut ke seluruh lapisan untuk memperbaiki bobot secara bertahap (Goodfellow, Bengio, & Courville, 2016). Dengan metode ini, jaringan dapat belajar dari kesalahan dan meningkatkan performa prediksi baik pada klasifikasi maupun regresi (Haykin, 2009).

D. Activation Function

Fungsi aktivasi adalah komponen penting dalam neural network yang menentukan output dari sebuah neuron berdasarkan input yang diterima. Fungsi ini memungkinkan jaringan untuk mempelajari hubungan non-linear pada data, sehingga MLP dapat melakukan klasifikasi atau regresi yang kompleks (Goodfellow, Bengio, & Courville, 2016). Tanpa fungsi aktivasi, jaringan hanya akan menjadi kombinasi linear dari input (Haykin, 2009). Beberapa fungsi aktivasi populer adalah Sigmoid, Tanh, ReLU, dan Softmax (Nielsen, 2015).

E. Klasifikasi

Klasifikasi adalah salah satu teknik dalam *supervised learning* yang bertujuan untuk mengelompokkan data ke dalam kategori berdasarkan label yang telah ditentukan. Dalam klasifikasi, model dilatih menggunakan dataset berlabel untuk mengenali pola dan menentukan kelas suatu data baru yang belum diketahui. Beberapa algoritma yang umum digunakan dalam klasifikasi meliputi Support Vector Machines (SVM), Normal Bayesian Classifier (NBC), K-Nearest Neighbor (KNN), Trees Gradient Boosted (GBT), Random Trees (RT), dan Artificial Neural Networks (ANN). Algoritma-algoritma ini membantu dalam memprediksi kelas suatu data berdasarkan fitur yang dimilikinya, memungkinkan aplikasi luas dalam berbagai bidang, termasuk deteksi penyakit, analisis gambar, dan sistem rekomendasi (Roihan, Sunarya, & Rafika, 2020).



F. Regresi

Linear regression merupakan salah satu algoritma yang memodelkan suatu persamaan untuk menghitung estimasi. Pada metode ini bertujuan untuk mencari pola pada nilai numerik, sehingga data yang dibutuhkan berupa data numerik agar dapat diolah dengan model algoritma ini. Linear regression memprediksi nilai data yang tidak diketahui dengan menggunakan nilai data lain yang terkait dan diketahui. Secara matematis memodelkan variabel yang tidak diketahui atau tergantung dan variabel yang dikenal atau independen sebagai persamaan linier (Mardiani, et al., 2023)

8.3 ALAT DAN BAHAN

A. Perangkat Keras

1. Laptop dengan minimal RAM 4 GB
2. Koneksi internet stabil

B. Perangkat Lunak

1. Browser (Chrome, Edge, Firefox, dsb)
2. Google Colab / Jupyter Notebook
3. Library ML (pandas, numpy, scikit-learn, matplotlib, dsb)

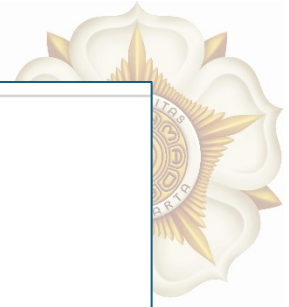
8.4 LANGKAH PERCOBAAN

1. Persiapkan IDE

Mahasiswa memulai percobaan dengan membuka Google Colab sebagai lingkungan pengembangan utama. Google Colab dipilih karena telah mendukung berbagai library machine learning yang dibutuhkan, Colab juga menyediakan resource komputasi berbasis CPU atau GPU secara gratis sehingga proses pelatihan model MLP dapat berjalan lebih optimal tanpa membutuhkan perangkat keras khusus

2. Instalasi dan Impor Library

Setelah lingkungan siap, mahasiswa mengimpor library yang diperlukan untuk manipulasi data, preprocessing, visualisasi, dan pembangunan model neural network. Library yang digunakan antara lain: pandas, numpy, matplotlib, seaborn, scikit-learn (MLPClassifier dan MLPRegressor), serta imbalanced-learn untuk metrik tambahan pada data tidak seimbang.



```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
from imblearn.metrics import sensitivity_specificity_support
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OrdinalEncoder
from sklearn.neural_network import MLPClassifier

import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Gambar 8.4.1 Impor Pustaka

3. Import dan Load Dataset Information

Dataset diambil dari sumber online (Kaggle/GitHub) dan dibaca menggunakan pandas.

Untuk percobaan:

1. Klasifikasi: dataset Bank Customer Churn digunakan untuk memprediksi pelanggan yang akan churn.
2. Regresi: dataset KC House Data digunakan untuk memprediksi harga rumah.

4. Exploratory Data Analysis

Pada tahap ini, mahasiswa melakukan eksplorasi awal terhadap dataset untuk memahami karakteristik data sebelum digunakan dalam model MLP.

5. Data Cleaning

Mahasiswa memeriksa keberadaan nilai hilang (missing values) dan memproses variabel kategorikal menggunakan teknik encoding (Label Encoding atau Ordinal Encoding). Fitur numerik kemudian dinormalisasi dengan StandardScaler untuk meningkatkan stabilitas pelatihan model MLP.

6. MLP untuk Klasifikasi

Mahasiswa membangun model MLPClassifier untuk memprediksi kategori target pada dataset churn. Proses mencakup:

1. Melihat beberapa baris data awal dan akhir head()

```
# read file
df = pd.read_csv('/content/drive/MyDrive/Penambangan Data Sem 6/BankChurners.csv')

# baris data dari atas
df.head()
```

	CLIENTNUM	Attrition_Flag	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	Income_Category
0	768805383	Existing Customer	45	M	3	High School	Married	\$60K - \$80K
1	818770008	Existing Customer	49	F	5	Graduate	Single	Less than \$40K
2	713982108	Existing Customer	51	M	3	Graduate	Married	\$80K - \$120K
3	769911858	Existing Customer	40	F	4	High School	Unknown	Less than \$40K
4	709106358	Existing Customer	40	M	3	Uneducated	Married	\$60K - \$80K

5 rows x 23 columns

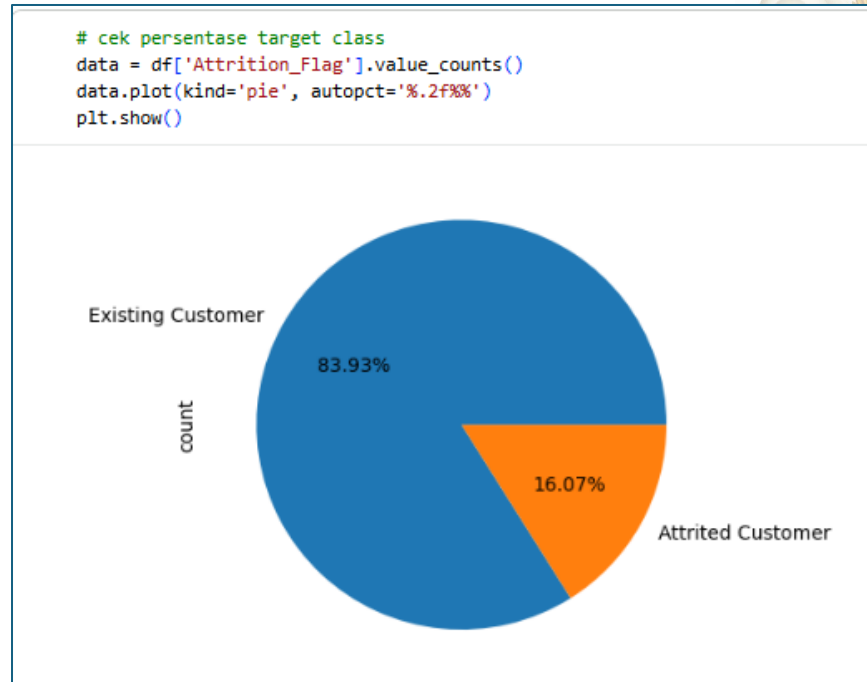
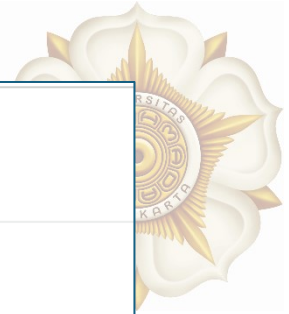
Gambar 8.4.2 Membaca dataset

2. Menganalisis distribusi variabel menggunakan histogram



Gambar 8.4.3 Menganalisis dataset

3. Mengecek proporsi target class (khusus dataset klasifikasi)



Gambar 8.4.4 Mengecek proporsi data

4. Mengecek missing value

```
# cek missing value
df.isnull().sum()
```

	0
CLIENTNUM	0
Attrition_Flag	0
Customer_Age	0
Gender	0

Gambar 8.4.5 Mengecek nilai null

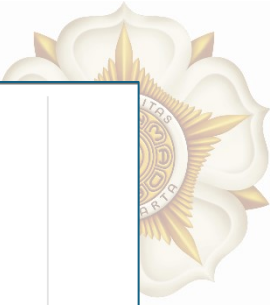
5. Mengidentifikasi atribut kategorikal dan numerik

```
# cek kategorikal atribut
df_X = df.drop(columns=['CLIENTNUM', 'Attrition_Flag'], axis=1)
df_y = df['Attrition_Flag']
cats = df_X.select_dtypes(include=['object', 'bool']).columns
print(cats)
```

```
Index(['Gender', 'Education_Level', 'Marital_Status', 'Income_Category',
      'Card_Category'],
      dtype='object')
```

Gambar 8.4.6 Mengindetifikasi atribut kategorikal dan numerik

6. Membagi data menjadi train dan test



```

# get X and y
df_X = df.drop(['CLIENTNUM', 'Attrition_Flag'],axis=1)
df_y = df[['Attrition_Flag']]

# label encoding for y.
# merubah nilai yg ada di y menjadi 0 atau 1.
# sebenarnya ini tidak diperlukan karena nilai y di dataset sudah 0 atau 1
le = LabelEncoder()
df_y = le.fit_transform(df_y['Attrition_Flag'])

# categorical encoding
# merubah categorical value menjadi numerical value
# bisa pakai label encoding, ordinal atau one hot encoding
cats = df_X.select_dtypes(include=['object', 'bool']).columns
cat_features = list(cats.values)
le = LabelEncoder()
for i in cat_features:
    le.fit(df_X[i])
    df_X[i] = le.transform(df_X[i])

# menyimpan X dan y menjadi numpy arrays
X = df_X.astype(float).values
y = df_y.astype(float)

# hold-out method, dibagi menjadi training dan testing set. 70% training, 30% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# scaling
scaler = StandardScaler().fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)

# mulai learning
model=MLPClassifier(max_iter=500)

# model=MLPClassifier(max_iter=1000, random_state=42, activation='logistic')
model.fit(X_train, y_train)

```

MLPClassifier

MLPClassifier(max_iter=500)

Gambar 8.4.7 Pembagian data training dan testing

```

# koef model
model.coefs_

```

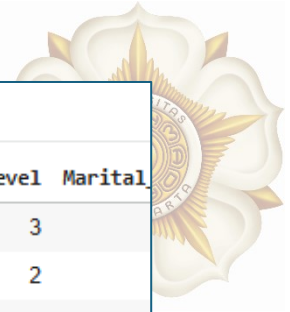
...

```

[ 0.25021812],
[-0.24980602],
[ 0.30144528],
[-0.17717679],
[-0.37201294],
[-0.2334015 ],

```

Gambar 8.4.8 Koefisien model



df_X					
	Customer_Age	Gender	Dependent_count	Education_Level	Marital
0	45	1	3	3	
1	49	0	5	2	
2	51	1	3	2	
3	40	0	4	3	
4	40	1	3	5	
...	...	...	...	...	...
10122	50	1	2	2	
10123	41	1	2	6	
10124	44	0	1	3	
10125	30	1	2	2	
10126	43	0	2	2	

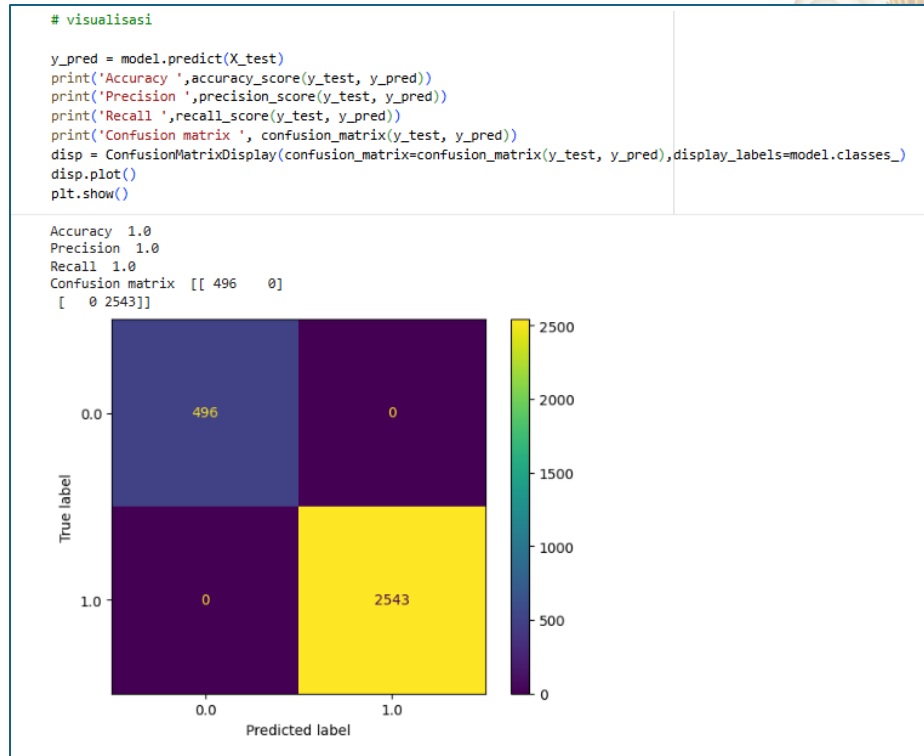
10127 rows × 21 columns

Gambar 8.4.9 Visualisasi Data

X_train[0]	
array([0.33519339, 1.05692234, -0.25437357, 0.4990858, -0.61710507, -0.56580813, -0.25591165, 0.25586758, -0.51407227, -0.33752656, -2.22835505, -0.05917368, 0.98686487, -0.14826286, -0.57038549, -0.90317904, -1.65490765, -1.12681489, -0.11093226, -0.43609419, 0.43609247])	

Gambar 8.4.10 Visualisasi Data

7. Mengevaluasi performa model menggunakan metrik seperti accuracy, precision, recall, serta confusion matrix



Gambar 8.4.11 Visualisasi Confusion Matrix

8. Melatih model dengan MLPClassifier

```
data = [{'Customer_Age':68, 'Gender':'F', 'Dependent_count':5, 'Education_Level':'Graduate',
'Marital_Status':'Single', 'Income_Category':'less than $40K', 'Card_Category':'Blue',
'Months_on_book':44, 'Total_Relationship_Count':6, 'Months_Inactive_12_mon':1,
'Contacts_Count_12_mon':2, 'Credit_Limit':8700, 'Total_Revolving_Bal':800, 'Avg_Open_To_Buy':7392,
'Total_Amt_Chng_Q4_Q1':1.541, 'Total_Trans_Amt':1200, 'Total_Trans_Ct':33, 'Total_Ct_Chng_Q4_Q1':3.714,
'Avg_Utilization_Ratio':0.10},
{'Customer_Age':50, 'Gender':'M', 'Dependent_count':2, 'Education_Level':'Graduate',
'Marital_Status':'Single', 'Income_Category':'$40K - $60K', 'Card_Category':'Blue',
'Months_on_book':38, 'Total_Relationship_Count':3, 'Months_Inactive_12_mon':2,
'Contacts_Count_12_mon':0, 'Credit_Limit':8077.0, 'Total_Revolving_Bal':1965, 'Avg_Open_To_Buy':6112.0,
'Total_Amt_Chng_Q4_Q1':0.637, 'Total_Trans_Amt':1329, 'Total_Trans_Ct':26, 'Total_Ct_Chng_Q4_Q1':0.444,
'Avg_Utilization_Ratio':0.243}]

df_input = pd.DataFrame(data)

# categorical encoding
# Use the original df_X columns to ensure correct feature order and presence
original_cols = df.drop(['CLIENTNUM', 'Attrition_Flag'], axis=1).columns
df_input = df_input.reindex(columns=original_cols, fill_value=0) # Reindex to match original columns, fill missing with 0 if necessary

cats = df_input.select_dtypes(include=['object', 'bool']).columns
cat_features = list(cats.values)

# Create a dictionary to store the fitted LabelEncoders for each categorical feature
label_encoders = {}

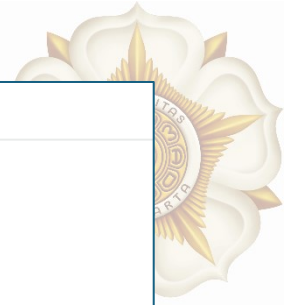
for i in cat_features:
    # Initialize a new LabelEncoder for each feature
    le = LabelEncoder()
    all_unique_values = pd.concat([df_X[i], df_input[i]]).astype(str).unique()
    le.fit(all_unique_values)
    label_encoders[i] = le
    df_input[i] = label_encoders[i].transform(df_input[i].astype(str))

# scaling
df_input = scaler.transform(df_input.astype(float).values)

result= model.predict(df_input)
for i in result:
    int_result = int(i)
    if (int_result == 0):
        decision = 'Existing Customer'
    elif (int_result==1):
        decision = 'Attrited Customer'
    else:
        decision = 'Not defined'
    print('Result prediction is ', decision)
```

Result prediction is Attrited Customer
Result prediction is Attrited Customer

Gambar 8.4.12 Melatih model MLP



df_input
array([[2.69329634e+00, 3.05998799e+00, 2.08111844e+00, 2.13632131e+00, 3.46696655e+00, 2.75905345e+00, 5.59066952e+00, 1.00082113e+00, 1.41213614e+00, -1.32245379e+00, -4.09541072e-01, 9.83216719e-03, -4.44011649e-01, -6.39776613e-03, 3.62111505e+00, -9.40555602e-01, -1.35950899e+00, 1.26325766e+01, -6.32215193e-01, -4.36229815e-01, -2.30899076e+00], [4.59304076e-01, 5.06305363e+00, -2.54373565e-01, 2.13632131e+00, 3.46696655e+00, -1.89575277e+00, 5.59066952e+00, 2.55867580e-01, -5.14072268e-01, -3.37526559e-01, -2.22835505e+00, -5.91736804e-02, 9.86864872e-01, -1.48262862e-01, -5.70385493e-01, -9.03179036e-01, -1.65490765e+00, -1.12681489e+00, -1.10932261e-01, -4.36229815e-01, -2.30899076e+00]])
cat_features
['Gender', 'Education_Level', 'Marital_Status', 'Income_Category', 'Card_Category']
result
array([1., 1.])

Gambar 8.4.13 Melihat hasil prediksi

7. MLP untuk Regresi

Mahasiswa membangun model **MLPRegressor** untuk memprediksi nilai kontinu (harga rumah). Langkah yang dilakukan:

1. Melihat beberapa baris data awal dan akhir head()

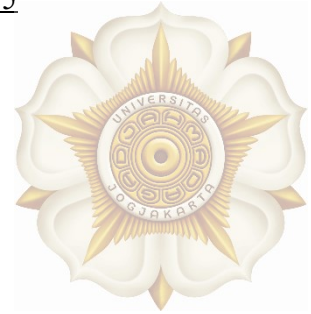
```
# read dataset
df = pd.read_csv('/content/drive/MyDrive/Penambangan Data Sem 6/kc_house_data.csv')
df.head()
```

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...	grade	sqft_above	sq
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650	1.0	0	0	...	7	1180.0	
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242	2.0	0	0	...	7	2170.0	
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000	1.0	0	0	...	6	770.0	
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000	1.0	0	0	...	7	1050.0	
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080	1.0	0	0	...	8	1680.0	

5 rows × 21 columns

Gambar 8.4.14 Membaca dataset

2. Mengecek missing value

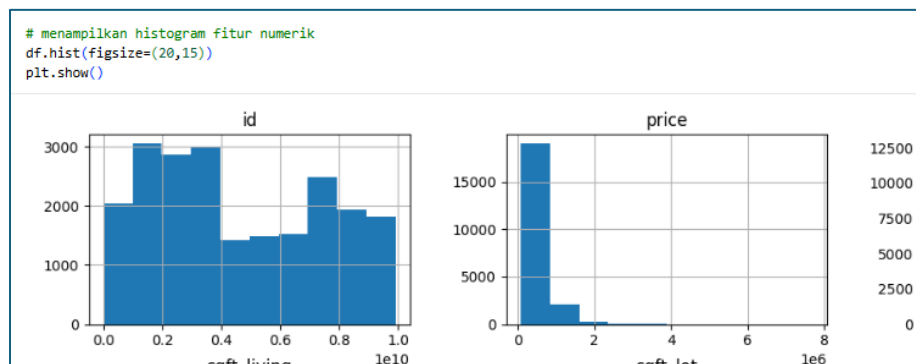


```
# cek missing values
df.isnull().sum()
```

***	0
id	0
date	0
price	0
bedrooms	0
bathrooms	0
sqft_living	0
sqft_lot	0
floors	0
waterfront	0
view	0
condition	0
grade	0

Gambar 8.4.15 Mengecek missing value

3. Menampilkan histogram fitur numerik



Gambar 8.4.16 Histogram fitur numerik

4. Mengukur performa model menggunakan R^2 (coefficient of determination)

```
# cek kategori atribut
df_X = df.drop(['id', 'date', 'price'], axis=1)
df_y = df[['price']]
cats = df_X.select_dtypes(include=['object', 'bool']).columns
print(cats)
```

Index([], dtype='object')

Gambar 8.4.17 Mengecek Performa



```

from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
import numpy as np

df_X = df.drop(['id', 'date', 'price'], axis=1)
df_y = df['price']

imputer = SimpleImputer(strategy='mean')
X = imputer.fit_transform(df_X)
y = df_y.astype(float).values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

reg = MLPRegressor()
reg.fit(X_train, y_train)

print('coef of determination training ', reg.score(X_train, y_train))
print('coef of determination testing ', reg.score(X_test, y_test))

print('prediction')
y_pred = reg.predict(X_test)
print(y_pred[:10])
print('real value')
print(y_test[:10])

coef of determination training  0.563641874546569
coef of determination testing  0.5484535001698863
prediction
[ 509811.91473618  733841.13263335 1094460.32414506 1358233.15019149
  632004.49719819  401595.71074248  638014.39483258  492009.77523119
  438309.65054247  315041.81873972]
real value
[ 365000.  865000. 1038000. 1490000.  711000.  211000.  790000.  680000.
  384500.  605000.]
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning:
warnings.warn(

```

Gambar 8.4.18 Mengecek performa

5. Mengukur performa model menggunakan R^2 (coefficient of determination) dan RMSE

```

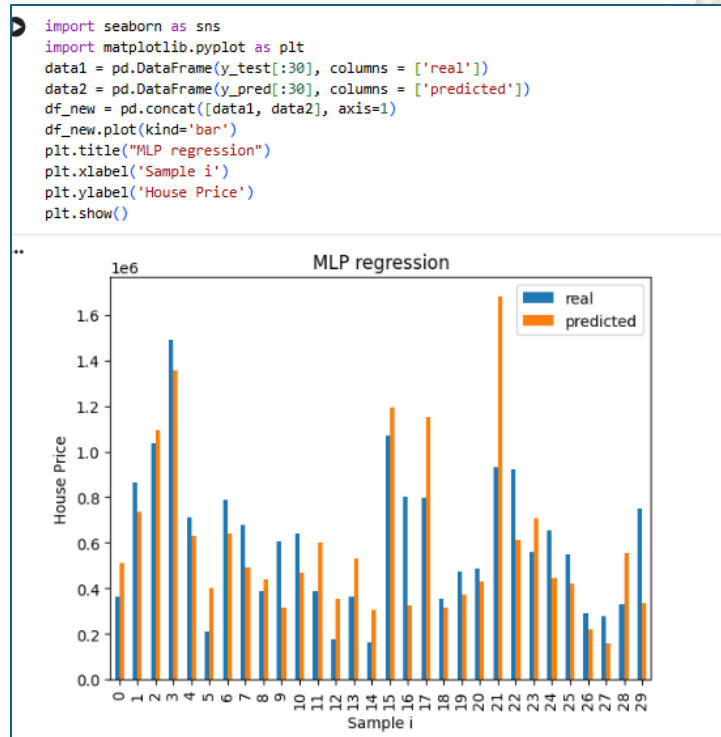
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
import numpy as np
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print('rmse : ', rmse)
print('r2 : ', r2)

rmse : 255319.8034520965
r2 : 0.5484535001698863

```

Gambar 8.4.19 Evaluasi model

6. Membuat visualisasi perbandingan prediksi vs nilai asli



Gambar 8.4.20 Visualisasi hasil prediksi

8.5 TUGAS DAN ANALISIS

1. Tugas 1

- Download dataset: [Bank Customer Churn Dataset](#) dan [kc_house_data](#)
- Jelaskan tujuan dataset ini
- Jelaskan feature dataset ini

2. Tugas 2

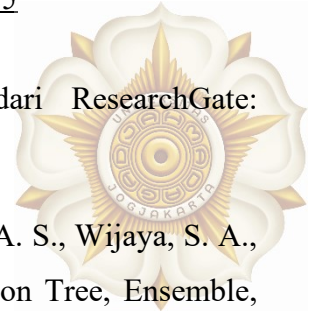
- Silahkan mencari dataset untuk klasifikasi, dan buat model MLP untuk klasifikasi
- Silahkan mencari dataset untuk regresi, dan buat model MLP untuk regresi
- Tampilkan masing-masing performa model klasifikasi dan regresi

8.6 REFERENSI

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Haykin, S. (2009). *Neural networks and learning machines* (3rd ed.). Pearson.

Nielsen, M. A. (2015). *Neural networks and deep learning: A textbook*. Determination Press.



Hukubun, A. J. (2022). *Neural Network*. Diambil kembali dari ResearchGate: https://www.researchgate.net/publication/365581579_Neural_Network

Mardiani, E., Rahmansyah, N., Ningsih, S., Lantana, D. A., Wirawan, A. S., Wijaya, S. A., & Putri, D. N. (2023). Komparasi Metode Knn, Naive Bayes, Decision Tree, Ensemble, Linear Regression Terhadap Analisis Performa Pelajar SMA. *INNOVATIVE: Journal Of Social Science Research*, 13880-13892.

Pardede, D., Hayadi, B. H., & Iskandar. (2022). KAJIAN LITERATUR MULTI LAYER PERCEPTRON: SEBERAPA BAIK PERFORMA ALGORITMA INI. *Journal of ICT Application and System*, 23-34.

Roihan, A., Sunarya, P. A., & Rafika, A. S. (2020). Pemanfaatan Machine Learning dalam Berbagai Bidang: Review paper. *IJCIT ((Indonesian Journal on Computer and Information Technology))*, 75-82.